Email Analysis Automation

Executive Technical Architecture, Workflow & GitHub Hosting Guide

PRODUCTION V1.0.0

Architecture & Systems Brief

1. Project Purpose & Executive Summary

The **Email Analysis Automation** platform is an autonomous, privacy-centric inbox triage intelligence assistant. It periodically inspects unread emails from Gmail via Google Cloud OAuth2, uses an LLM through **Ollama** for context-aware classification and action-item extraction, and delivers concise, actionable headline briefings directly to a private **Telegram** chat.

2-Hour Autonomous Cycle

Scheduled background monitoring queries only messages within rolling 120-minute windows, avoiding stale duplicates.

AI Executive Triage

Separates urgent academic/work/billing items from promotional newsletters, cutting inbox noise by over 90%.

Interactive Telegram Bot

Instant on-demand `trigger` executions and real-time `status` monitoring protected by strict Chat ID security.

2. Complete Technology Stack Matrix

The system is built on a modular, lightweight modern JavaScript/TypeScript stack designed to run locally, on a serverless task worker, or inside a containerized server:

COMPONENT LAYER	TECHNOLOGY / TOOL	ROLE & ARCHITECTURAL SIGNIFICANCE
Runtime & Language	Node.js (v18+) TypeScript (v5.5)	Native ES Module architecture handling non-blocking async network I/O. Strict TypeScript typechecking for Trigger.dev task schemas, environment validation, and error boundaries.
Email Ingestion & Auth	Gmail REST API v1 Google OAuth 2.0	Performs query filtering (`q=after:{epoch}`) with automated OAuth2 refresh token rotation. Decodes nested multipart MIME bodies (`text/plain` & `text/html`) via Base64URL parsing.
AI Inference Engine	Ollama HTTP API gemma4:31b-cloud / llama3.2	Fast, local/private LLM inference via `/api/chat`. Employs prompt engineering for triage classification, generating bold headline bullets, explicit action items, and fallback headline digests.
Notification & Interface	Telegram Bot API BotFather & Long-Polling	Delivers HTML-formatted push notifications directly to the user's private chat. Runs a continuous long-polling listener (`getUpdates`) for interactive commands with zero-trust Chat ID checks.
Task Orchestration	Trigger.dev (v4.5 SDK) Autonomous Daemon	Dual scheduling: Cloud-native task queuing with cron expressions (`0 */2 * * *`) and run telemetry, complemented by a zero-dependency local daemon (`setInterval`) for standalone execution.
Version Control & CI	Git & GitHub GitHub Actions CI	Source control repository, automated typecheck workflows, and protected environment secrets (OAuth tokens, bot keys) keeping credentials strictly excluded from public commits.

Privacy-First Architecture: Raw email contents are processed strictly in-memory or on local LLM runtimes. No personal email data or credentials are ever written to third-party analytic databases or shared across public cloud endpoints.

System Workflows & Flowcharts

Visualizing the Ingestion, Triage, and Dispatch Pipeline

PROCESS FLOW

Architecture & Systems Brief

3. End-to-End Pipeline Workflow

The lifecycle of an email analysis cycle executes across six coordinated stages: trigger initiation, authenticated token exchange, lookback query fetch, MIME extraction, LLM triage, and formatted Telegram push notification.

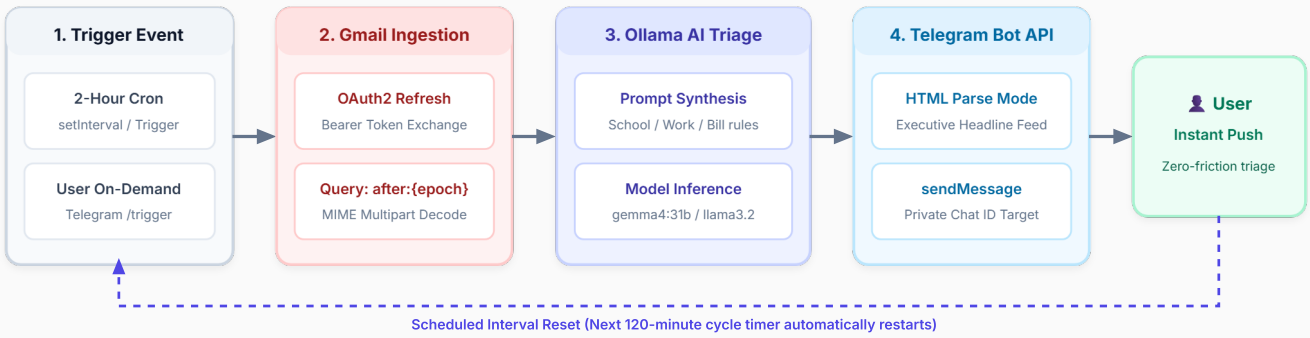


Figure 1: Complete Asynchronous End-to-End Email Analysis & Briefing Pipeline

4. Telegram Interactive Command & Security Architecture

The interactive listener operates continuously using non-blocking long-polling (`getUpdates`). Incoming requests are filtered through a strict zero-trust security gate before dispatching commands:

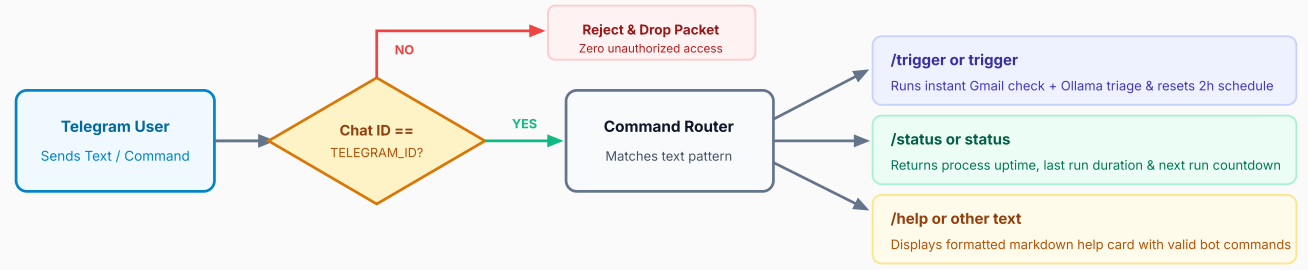


Figure 2: Zero-Trust Security Validation & Command Routing Flow in the Telegram Listener

HIGH PRIORITY TRIAGE

Identifies: Academic deadlines, university course updates, work client requests, and expiring subscriptions/billing.
Output: 1-line bold headline bullet + clear, explicit action item.

LOW PRIORITY & FALLBACK

Aggregates: Marketing promotions, social pings, newsletters condensed into a single summary line.
Resilience: Automatic deterministic regex fallback if Ollama is offline.

GitHub Hosting & Deployment

Repository Setup, Security Boundaries, CI/CD and Production Hosting

DEVOPS & HOSTING

Architecture & Systems Brief

5. GitHub Repository Hosting & Directory Structure

Hosting this project on GitHub facilitates version control, automated testing, and deployment orchestration. Because email access credentials and bot tokens are critical assets, a strict zero-leakage security model is enforced:

- ✓ **Protected Credentials:** `.gitignore` strictly excludes `.env`, `.env.local`, `node_modules/`, and `.trigger/` to prevent exposing Google OAuth secrets.
- ✓ **Template Provided:** `.env.example` acts as the canonical configuration contract with placeholder keys for public cloning.
- ✓ **Continuous Integration:** Automated GitHub Actions workflow checks TypeScript types (`tsc --noEmit`) on every commit.
- ✓ **Branch Strategy:** Maintain a stable `main` branch for deployments and feature branches for additions.

```
# Git Repository Directory Layout
email-analysis/
├── scripts/
│   ├── instant-check.mjs      # Triage engine
│   ├── telegram-listener.mjs # Bot & scheduler
│   └── get-gmail-token.mjs    # OAuth helper
├── src/trigger/               # Trigger tasks
├── .env.example               # Safe template
├── package.json               # NPM scripts
└── trigger.config.ts          # Trigger config
```

6. Setting Up Remote Repository on GitHub

To initialize and push the repository to GitHub under your user account:

```
# 1. Initialize Git repository and stage tracked files (verifying .gitignore protects .env)
git init
git add .
git commit -m "feat: initial commit of email analysis and triage automation"

# 2. Link your GitHub remote repository and push main branch
git remote add origin https://github.com/<your-username>/email-analysis-automation.git
git branch -M main
git push -u origin main
```

7. GitHub Actions CI & Production Hosting Models

The project supports three distinct hosting strategies depending on your operational requirements:

HOSTING STRATEGY	SETUP & EXECUTION MECHANISM	IDEAL USE CASE & STRENGTHS
1. Local Daemon / PM2 Self-Contained	Run as a background system daemon using PM2 or native Node: `pm2 start scripts/telegram-listener.mjs --name email-bot`	Zero cloud cost. Directly connects to local **Ollama GPU inference** without exposing internal networks.
2. VPS / Docker Server Always-On Cloud	Clone repo onto Ubuntu/Debian server or Raspberry Pi. Add `.env` file. Run as a `systemd` service or Docker container.	Guarantees 24/7 reliability and uptime without requiring your personal workstation to remain awake.
3. Trigger.dev Cloud Serverless Tasks	Deploy with `npx trigger deploy`. Configure OAuth and Bot keys securely in the Trigger.dev Cloud Dashboard.	Serverless scalability, automatic retries, execution observability dashboards, and zero server maintenance.

8. Rapid 3-Step Setup Cheatsheet

1. Clone & Install

```
git clone <repo-url>
npm install
```

2. Authenticate

```
cp .env.example .env
npm run auth:gmail
```

3. Launch Bot

```
npm run schedule
npm run check:instant
```

🚀 **Production-Ready Automation:** The project combines Google Cloud OAuth2 authentication, fast LLM inference via Ollama, zero-trust Telegram control, and flexible deployment models ready for hosting.